

PROJECT REPORT

UART- Universal Asynchronous Receiver Transmitter

Student Name :

B.Roshan Raghavendra

1. Abstract

This project presents the design and implementation of a Universal Asynchronous Receiver Transmitter (UART) on the Basys3 FPGA using Verilog HDL. The UART module enables serial communication between digital devices and includes features such as a baud rate generator, transmitter, receiver, finite state machine (FSM) control, and even parity checking for error detection. The complete design was synthesized, simulated, and tested on hardware using Xilinx Vivado, demonstrating correct data transmission and reception.

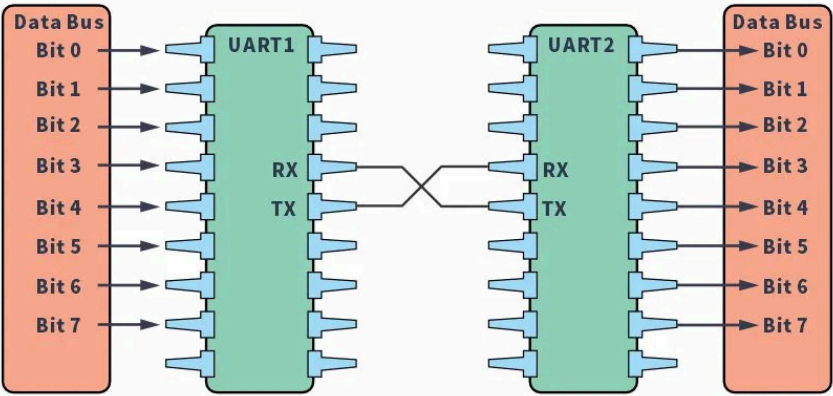
2. Introduction

UART is a widely used protocol for asynchronous serial communication in embedded systems. It allows two devices to exchange data over a simple two-wire interface (TX and RX) without needing a shared clock. The main objective of this project was to design and verify a working UART module that supports both transmission and reception with even parity checking on the Basys3 FPGA development board.

3. UART Format

*UART data is transmitted in a structured frame that includes a **start bit**, **data bits**, an **optional parity bit**, and one or more **stop bits**. Transmission begins with a start bit (logic 0), followed by typically 8 data bits (LSB first), then a parity bit (for basic error detection), and ends with one or two stop bits (logic 1). This format enables asynchronous communication without requiring a shared clock between devices.*

UART Communication



4. Design Description

Top module

```
module uart_debug_top (
    input clk,
    input rst,
    input rx,
    input btnT,
    input [7:0] sw,
    output tx,
    output reg [7:0] led
);

    wire [7:0] rx_data;
    wire      rx_valid;
    wire      rx_error;

    wire      tx_busy;

    reg [7:0] tx_data = 8'b0;
    wire transmit;

    debounce d1(.pb_1(btnT),.clk(clk),.pb_out(transmit));

    // UART Transmitter (TX)
    uart_tx #(.CLKS_PER_BIT(868)) uart_tx_inst (
        .clk(clk),
        .rst(rst),
        .tx_start(transmit),
        .data_in(tx_data),
        .tx(tx),
        .tx_busy(tx_busy)
    );
```

```

// UART Receiver (RX)
uart_rx #(.CLKS_PER_BIT(868)) uart_rx_inst (
    .clk(clk),
    .rst(rst),
    .rx(rx),
    .data_out(rx_data),
    .rx_valid(rx_valid),
    .rx_error(rx_error)
);

always @(posedge clk or posedge rst) begin
    if (rst) begin
        led <= 8'b0;
    end else if (rx_valid) begin
        led <= rx_data;
    end
end

always @(posedge clk or posedge rst) begin
    if (rst) begin
        tx_data <= 8'b0;
    end else begin
        tx_data <= sw;
    end
end

endmodule

```

Transmitter module

```
module uart_tx (
    input        clk,
    input        rst,
    input        tx_start,
    input  [7:0] data_in,
    output reg    tx,
    output reg    tx_busy
);

    parameter CLKS_PER_BIT = 10'd868; // 100 MHz clock for 115200 baud

    reg [3:0] bit_idx = 0;
    reg [13:0] clk_cnt = 0;
    reg [10:0] shift_reg = 11'b0;
    reg        sending = 0;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            tx            <= 1;
            tx_busy       <= 0;
            sending       <= 0;
            clk_cnt       <= 0;
            bit_idx       <= 0;
            shift_reg     <= 0;
        end

        else begin
            if (tx_start && !sending) begin
                shift_reg <= {1'b1, ^data_in, data_in, 1'b0};
                sending   <= 1;
                tx_busy   <= 1;
                clk_cnt   <= 0;
                bit_idx   <= 0;
            end else if (sending) begin
                if (clk_cnt < CLKS_PER_BIT - 1)
                    clk_cnt <= clk_cnt + 1;
                else begin
                    clk_cnt <= 0;
                    tx <= shift_reg[bit_idx];
                    bit_idx <= bit_idx + 1;

                    if (bit_idx == 10) begin
                        sending <= 0;
                        tx_busy <= 0;
                        tx <= 1;
                    end
                end
            end
        end
    end
endmodule
```

Receiver Module

```
module uart_rx (
    input      clk,
    input      rst,
    input      rx,
    output reg [7:0] data_out,
    output reg  rx_valid,
    output reg  rx_error
);
    parameter CLKS_PER_BIT = 868;

    reg [3:0] bit_idx = 0;
    reg [13:0] clk_cnt = 0;
    reg [10:0] shift_reg = 0;
    reg      receiving = 0;
    reg      rx_sync = 1;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            bit_idx    <= 0;
            clk_cnt    <= 0;
            receiving   <= 0;
            shift_reg   <= 0;
            data_out    <= 0;
            rx_valid    <= 0;
            rx_error    <= 0;
            rx_sync     <= 1;
        end
        else
            begin
                rx_sync <= rx;
                rx_valid <= 0;

                if (!receiving)
                    begin
                        if (rx_sync == 0)
                            begin
                                receiving <= 1;
                                clk_cnt <= CLKS_PER_BIT / 2;
                                bit_idx <= 0;
                            end
                        end
                    else
                        begin
                            if (clk_cnt > 0)
                                clk_cnt <= clk_cnt - 1;
                            else
                                begin
                                    clk_cnt <= CLKS_PER_BIT - 1;
                                    shift_reg[bit_idx] <= rx_sync;
                                    bit_idx <= bit_idx + 1;

                                    if (bit_idx == 10)
                                        begin
                                            receiving <= 0;
                                            if (shift_reg[0] != 0 || rx_sync != 1 || (^shift_reg[8:1] != shift_reg[9]))
                                                begin
                                                    rx_error <= 1;
                                                end
                                            else
                                                begin
                                                    data_out <= shift_reg[8:1];
                                                    rx_valid <= 1;

                                                    rx_error <= 0;
                                                end
                                            end
                                        end
                                    end
                                end
                            end
                        end
                    end
                end
            end
        end
    endmodule
```

Debouncing module

```
module debounce(input pb_1,clk,output pb_out);
wire slow_clk;
wire Q1,Q2,Q2_bar,Q0;
clock_div u1(clk,slow_clk);
my_dff d0(slow_clk, pb_1,Q0 );
my_dff d1(slow_clk, Q0,Q1 );
my_dff d2(slow_clk, Q1,Q2 );
assign Q2_bar = ~Q2;
assign pb_out = Q1 & Q2_bar;
endmodule

module clock_div(input Clk_100M, output reg slow_clk

);
reg [26:0]counter=0;
always @(posedge Clk_100M)
begin
    counter <= (counter>=7811)?0:counter+1;
    slow_clk <= (counter < 3905)?1'b0:1'b1;
end
endmodule

module my_dff(input DFF_CLOCK, D, output reg Q);

always @ (posedge DFF_CLOCK) begin
    Q <= D;
end

endmodule
```

5. Features and Enhancements

- Implements UART protocol for asynchronous serial communication.
- Baud rate generator produces accurate timing (9600 bps) from 100 MHz clock.
- Button edge detection used to trigger data transmission.
- Switches are used to manually input 8-bit data, which is stored in registers.
- Shift registers used for both transmission and reception of serial data.
- Even parity bit is generated and verified for basic error checking.
- Simple and resource-efficient design without using FSMs.

6. implementation details

- Switches are used for transmitting the data
- LEDs are used for receiving
- Even parity and Error detection are also given to LEDs
- Debouncing is used to overcome bouncing character of the hardware buttons.

7. Challenges

- **Timing Issues:** Ensuring correct baud rate division from a 100 MHz clock required careful calculation and debugging.

- **Edge Detection:** Reliable button press detection without multiple triggers was tricky and needed debounce logic.
- **Parity Checking:** Implementing and verifying even parity logic for both TX and RX required extra care during testing.

8. Conclusion

The UART project was successfully implemented on the Basys3 FPGA using Verilog and shift registers. It enabled reliable serial communication with features like manual input using switches, edge-triggered transmission, and even parity checking. The design was tested through simulation and on hardware, and it worked as expected. This project helped reinforce important concepts like baud rate generation, serial data handling, and hardware debugging, while offering a hands-on understanding of UART communication.

